



Department of Computer & Software Engineering

College of E&ME, NUST, Rawalpindi

Project

Names of Members:
Mohammad Usman
Mohammad Husnain
Abdullah Aqeel

Table of Contents

Section No.	Section Title	Link
1.	Objectives	Section 1
2.	Problem Statement	Section 2
3.	Implementation Overview	Section 3
	Architectural Key Features	Section 3.1
4.	Theory of Multi-Cycle Architecture	Section 4
5.	List of Module Names	Section 5
6.	Instructions for the M types	Section 6
	FSM	Section 6.1
7.	The Control unit	Section 7
8.	Simulation	Section 8
	Instructions and their Results	Section 8.1
9.	The RTL Diagram	Section 9
10.	The Multiplication module	Section 10
11.	Conclusion	Section 11

1. Objectives

The primary goal of this project is to extend the capabilities of an integer-only RV32I core to support high-speed arithmetic operations through hardware acceleration. Specific objectives include:

- Hardware Design: Developing a dedicated multiplier module to perform efficient multiplication, avoiding slow software-based iterative methods.
- System Integration: Integrating the multiplier into a multi-cycle processor architecture using a separate data path to maintain structural integrity.
- Verification: Ensuring the co-processor interfaces cleanly with the existing control unit and register file to correctly execute M-extension instructions.

2. Problem Statement

Multiplication is a complex operation involving multiple partial product generations and accumulations. In a standard RV32I core, these operations must be simulated via software, which is computationally expensive and slow.

The challenge is to design a hardware unit that:

- Handles the increased complexity of multiplication without degrading the performance of the existing RISC-V pipeline.
- Provides the high-speed arithmetic required for modern applications like Digital Signal Processing (DSP), machine learning, and cryptography.

- Correctly manages data flow between the existing register file and the new multiplication module within the multi-cycle timing constraints.

Implementation Overview

As per the provided schematic, the implementation utilises a multi-cycle architecture where the multiplier operates on a separate path from the standard ALU.

Architectural Key Features:

- Dedicated Multiplication Module: A standalone unit (multiplication_module) is instantiated to handle the 32-bit operands.
- Path Separation: To prevent bottlenecks, a mul_mux and Mul_output_mux are used to select between standard ALU results and multiplication results before they are written back to the register file.
- Control Integration: The central Control_Unit and alu_control are modified to decode M-extension instructions, routing data to the multiplier when specific opcodes are detected.
- Output Handling: The design supports multiple outputs (e.g., mul_hi and mul_lo) to accommodate the 64-bit result typically generated by a 32x32 multiplication.

Theory of Multi-Cycle Architecture

The multi-cycle architecture is a design approach where a single instruction is divided into several smaller steps, each executed in a distinct clock cycle. This method optimizes hardware by allowing different functional units to be reused during the execution of a single instruction. For example, a single memory unit can be used for both fetching instructions and accessing data, and a single ALU can be used for both address calculation and arithmetic operations.

- Instruction Partitioning: Every instruction is broken down into sequences such as Fetch, Decode, Execute, Memory, and Write-Back.
- Clock Cycle Efficiency: The clock period is determined by the delay of the slowest individual functional unit rather than the entire instruction path, often leading to a higher clock frequency than single-cycle designs.
- Resource Sharing: By spreading execution over multiple cycles, the design reduces the total amount of hardware required, as units like the ALU and Memory do not need to be duplicated.
- Intermediate Storage: Registers are placed between functional units to "latch" or hold the output of one cycle so it can be used as the input for the next cycle.
- Control Complexity: A Finite State Machine (FSM) is used to track which stage an instruction is in and to assert the correct control signals for each specific cycle.

List of Module Names

Based on your implementation and the provided schematic, the system consists of the following key modules:

- **PC Register (pc_reg**): Maintains the address of the current instruction being processed.
- **Unified Memory**: A single-port memory module used for storing both the instruction set and the data.
- **Instruction Register (instr_reg):** Holds the instruction fetched from memory to ensure it remains stable during decoding and execution.
- **Control Unit (Control_Unit):** The primary FSM that generates control signals for the datapath based on the current state and opcode.
- **ALU Control (alu_control):** Decodes the specific arithmetic/logic operation required based on the instruction's funct fields.
- **Register File (reg_file):** Contains the 32 general-purpose RISC-V registers (x0-x31) for operand storage.
- **Immediate Generator (imm_gen):** Extracts and sign-extends immediate values from the instruction for operations like addi or memory offsets.
- **ALU (ALU):** Performs the core integer arithmetic and logical operations for the RV32I base set.
- **Multiplication Module (multiplication_module):** The custom hardware co-processor designed to handle M-extension instructions.
- **ALU Source 1 Mux (alu_src1):** Selects the first operand for the ALU, typically choosing between the current Program Counter (PC) for address incrementing or a register value (rs1) for standard arithmetic.
- **ALU Source 2 Mux (alu_src2):** Selects the second operand, switching between a register value (rs2), a sign-extended immediate from the imm_gen module, or a constant (like 4) for PC updates.

- **Multiplication Mux (mul_mux):** A specialised selector for your co-processor that determines if the data sent to the next stage should come from the standard ALU or the new multiplication_module.
- **ALU Output Mux (Mul_output_mux):** Finalises the path for the computed result, ensuring that during an "M-extension" instruction, the multiplier's output is prioritised over the ALU's output for the write-back stage.
- **Address Source Mux (address_source_mux):** Controls the memory input, selecting whether the memory address comes from the PC (during Instruction Fetch) or from the ALU output (during a Load/Store operation).
- **Write-Back Mux**: Chooses the final data to be written into the register file, often selecting between the ALU/Multiplier result, data read from memory (MDR), or the PC.

Instructions for the M types

Standard Extensions

RV32M Multiply Extension

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
mul	MUL	R	0110011	0x0	0x01	rd = (rs1 * rs2)[31:0]
mulh	MUL High	R	0110011	0x1	0x01	rd = (rs1 * rs2)[63:32]
mulsu	MUL High (S) (U)	R	0110011	0x2	0x01	rd = (rs1 * rs2)[63:32]
mulu	MUL High (U)	R	0110011	0x3	0x01	rd = (rs1 * rs2)[63:32]
div	DIV	R	0110011	0x4	0x01	rd = rs1 / rs2
divu	DIV (U)	R	0110011	0x5	0x01	rd = rs1 / rs2
rem	Remainder	R	0110011	0x6	0x01	rd = rs1 % rs2
remu	Remainder (U)	R	0110011	0x7	0x01	rd = rs1 % rs2

FSM

The main Control FSM functions as the central logic for the multicycle processor, orchestrating the transition from standard instruction states like Fetch (S0) and Decode (S1) into specialised execution branches. While basic R-type and I-type instructions complete in a single execution cycle before moving to Write-Back (S7), the addition of the ExecM (S11) state introduces a high-latency branch designed for iterative multiplication and division. In this state, the FSM effectively stalls and enters a synchronisation loop, holding the current state constant until the external multiplication module asserts the mult_ready signal, signalling that the 32-bit iterative calculation is complete.

Once the multiplier finishes its process, the FSM transitions to the shared ALU Write-Back (S7) state to commit the result to the register file. Unlike the load/store path, which must navigate through separate Memory Address (S2) and Memory Read (S3) cycles, the multiplication branch utilises the data directly from the multiplication module's hi or lo outputs based on the instruction's funct3 field. This vertical flow ensures that, regardless of the complexity of the operation—whether a simple addition, a multi-cycle memory access, or a 32-cycle division—every instruction path eventually converges to either a write-back state or a direct return to the Fetch state to begin the next cycle.

Changes to the data path

Multiplication Module

This is the custom hardware co-processor designed to handle the high-speed arithmetic requirements of the M-extension[1].

Functionality: It performs iterative 32-bit multiplication and division, which avoids the slow software-based methods used in a standard RV32I core

Inputs: It receives two 32-bit operands (data1 and data2) from the register file and control signals from the alu_control.

Outputs: It generates a 64-bit result partitioned into hi and lo 32-bit segments to accommodate the full product of a 32 times 32 operation

Status Signal: It asserts the mult_ready signal once the 32-cycle calculation is complete, allowing the main FSM to proceed

Control Unit & PC Stall Logic

To manage the high latency of hardware multiplication, the Control Unit was modified to handle timing constraints.

The ExecM (S11) State: A specialised execution branch was added to the main FSM specifically for M-extension instructions

PC Stalling: In the S11 state, the FSM enters a synchronisation loop where the Program Counter (PC) and current state are held constant[8].

Flow Control: The PC does not increment, and the instruction does not move to the Write-Back stage until mult_ready is high. This ensures the processor "stalls" and waits for the iterative calculation to finish.

Multiplication Mux (mul_mux)

This specialised selector manages the internal data routing within the co-processor path.

Selection Logic: It determines which part of the 64-bit result is required by the specific instruction.

Funct3 Dependency: Based on the funct3 field (e.g., 0x0 for mul or 0x1 for mulh), this mux selects between

the hi and lo 32-bit outputs of the multiplication module

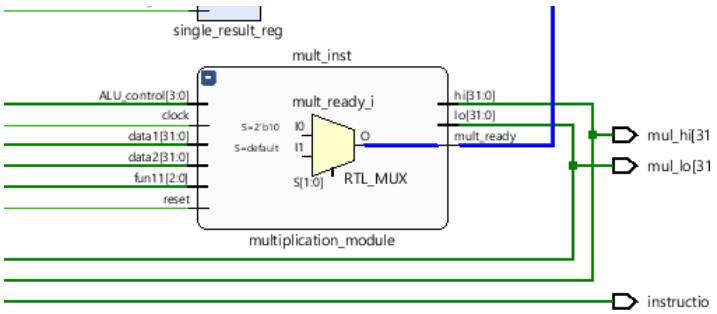
Final Multiplication Output Mux (Mul_output_mux)

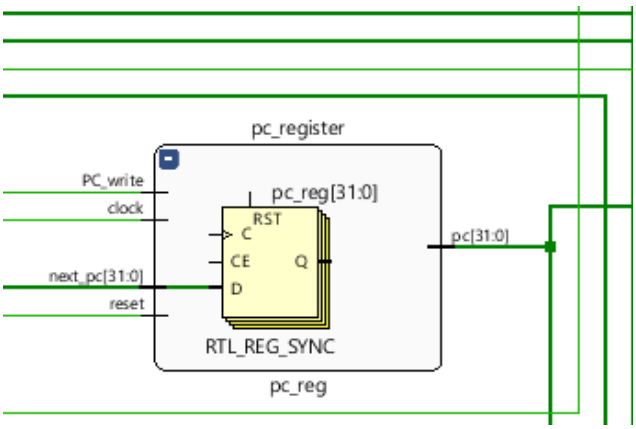
This mux is the final gatekeeper before the data reaches the Register File write-back stage

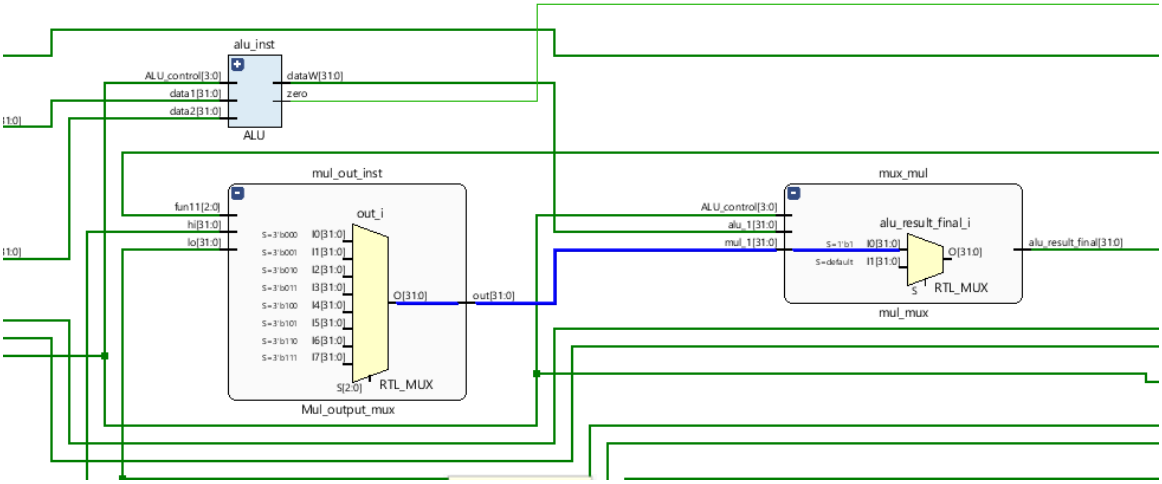
Path Integration: It sits at the junction of the standard ALU path and the new multiplication path

Priority Switching: When an M-extension opcode is detected, this mux prioritises the output of the multiplication_module over the standard ALU_result.

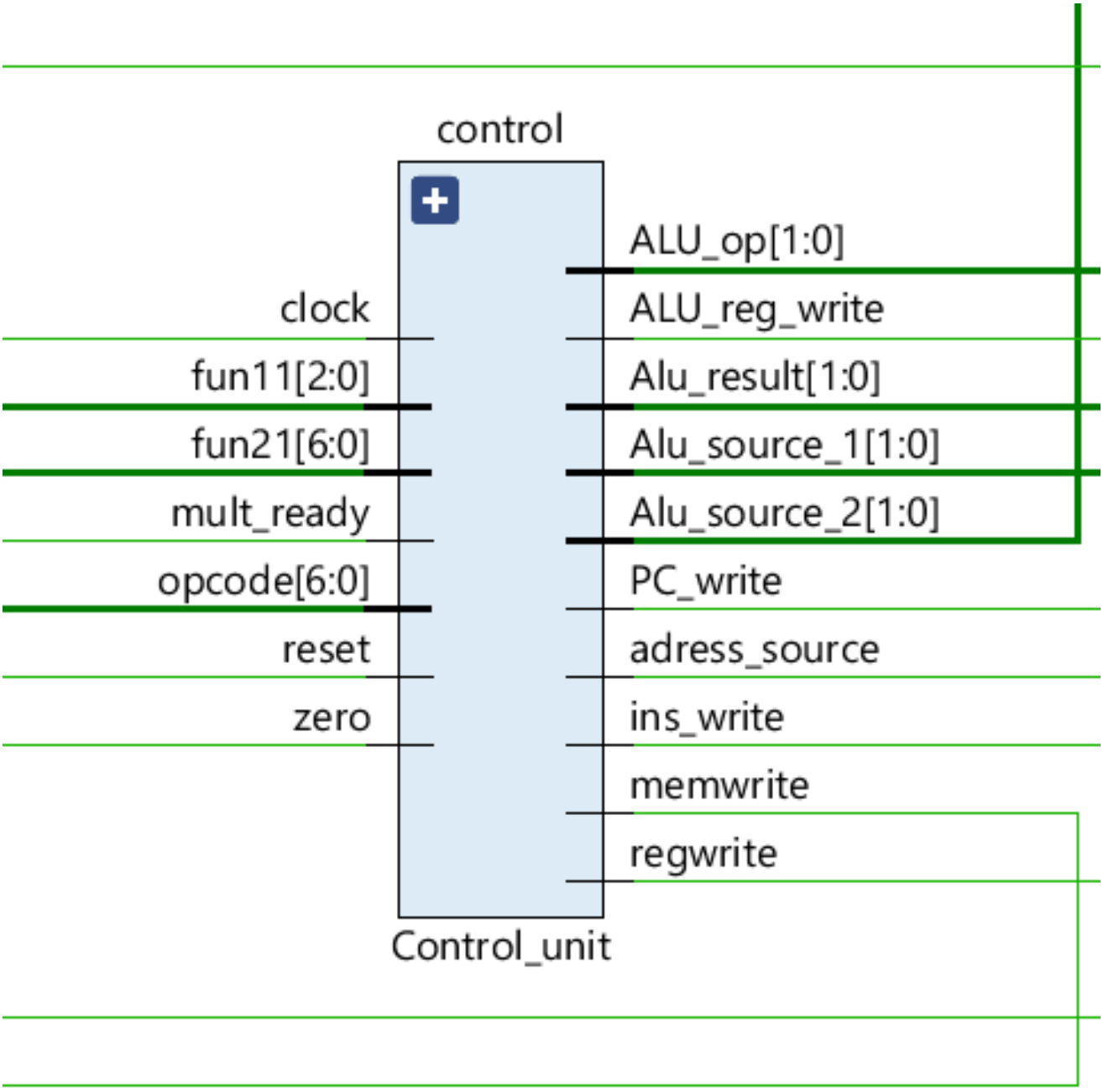
Final Result: It ensures that during the Write-Back stage (S7), the correct calculated value is committed to the destination register rd



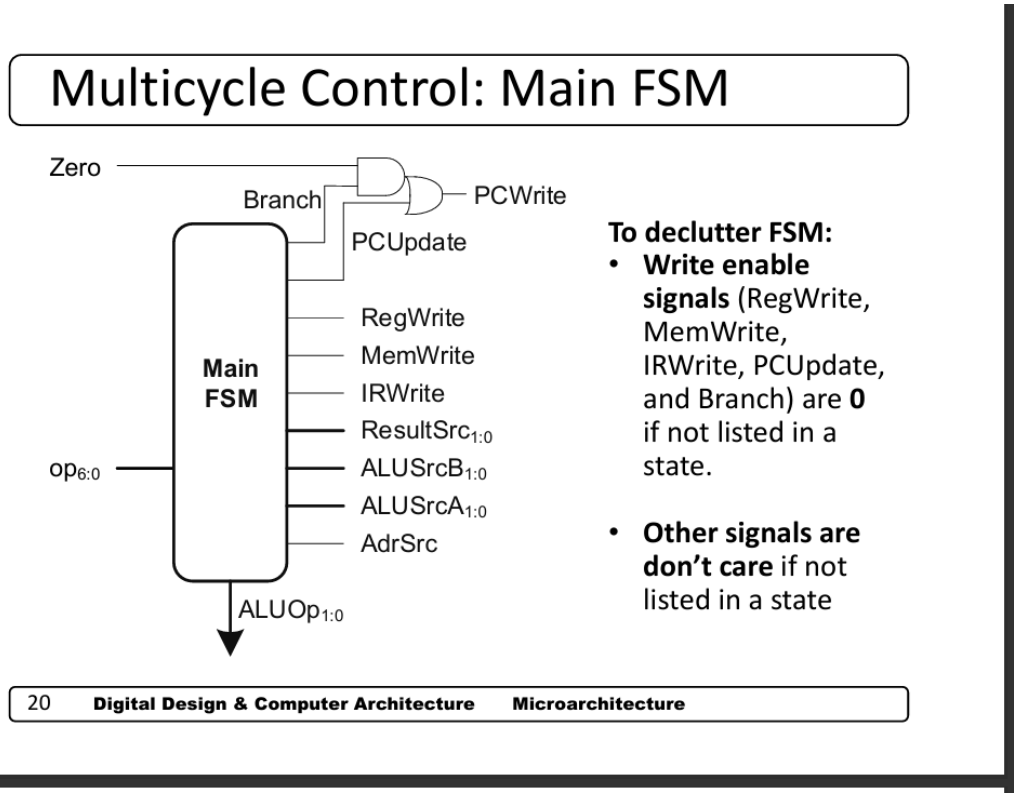




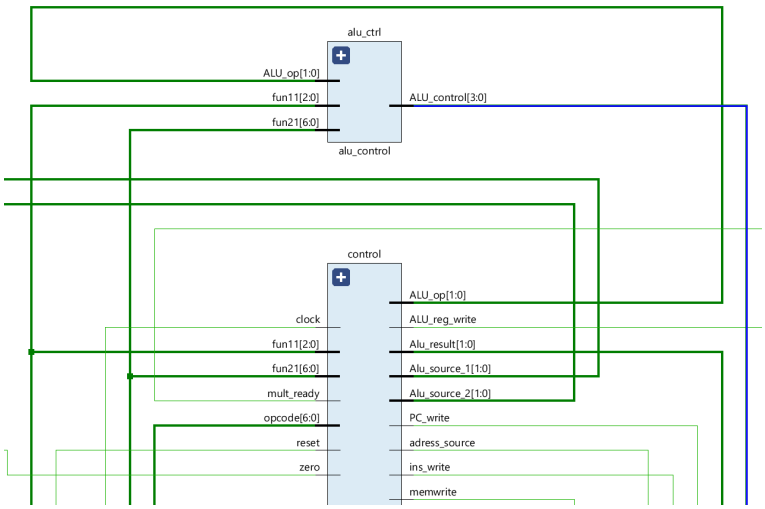
The Control unit



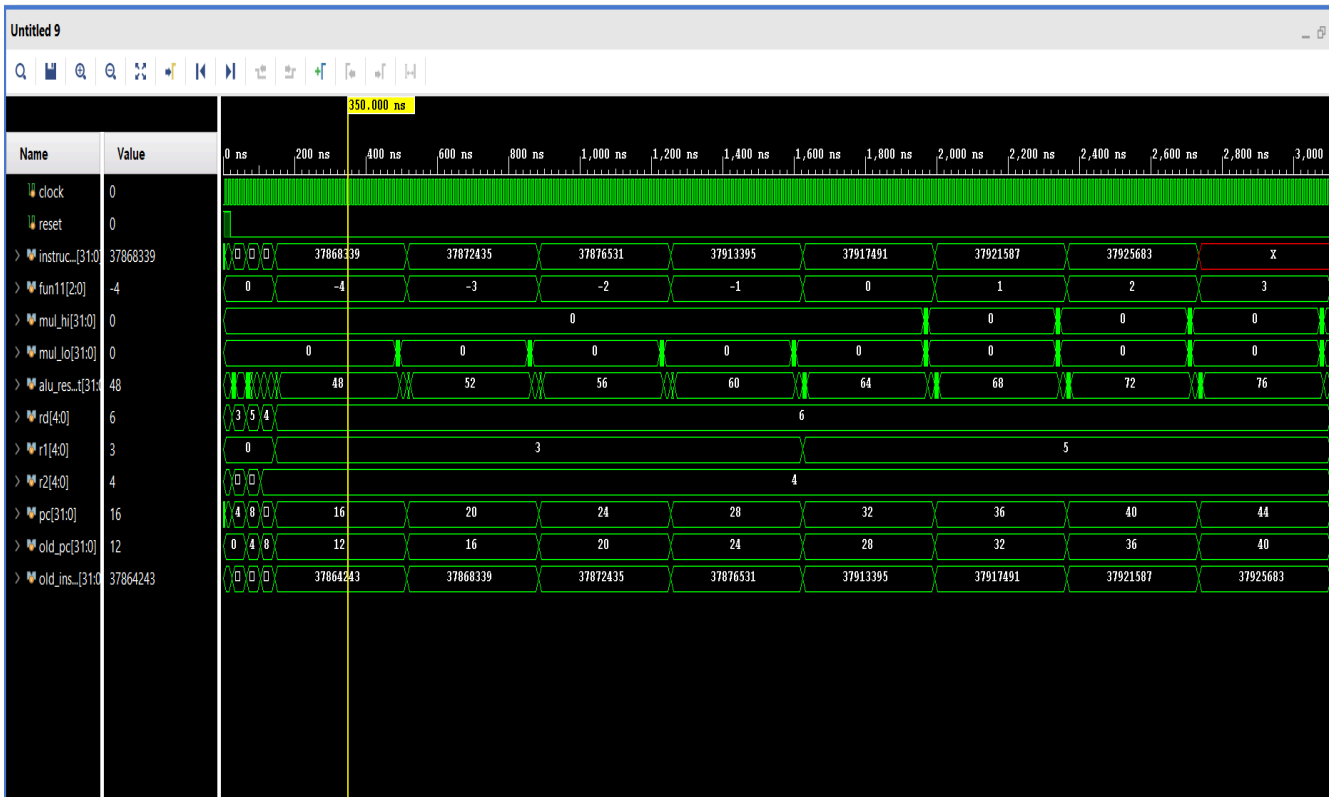
- It is based on



The main Control was connected with an AlU control, which is



Simluation



Instructions and their Results

0x00A00193

0xFF600293

0x00400213

0x0241C333

0x0241D333

0x0241E333

0x0241F333

0x02428333

0x02429333

0x0242A333

0x0242B333

<u>This corresponds to</u>

addi x3, x0, 10

addi x5, x0, -10

addi x4, x0, 4

div x6, x3, x4

divu x6, x3, x4

rem x6, x3, x4

remu x6, x3, x4

mul x6, x5, x4

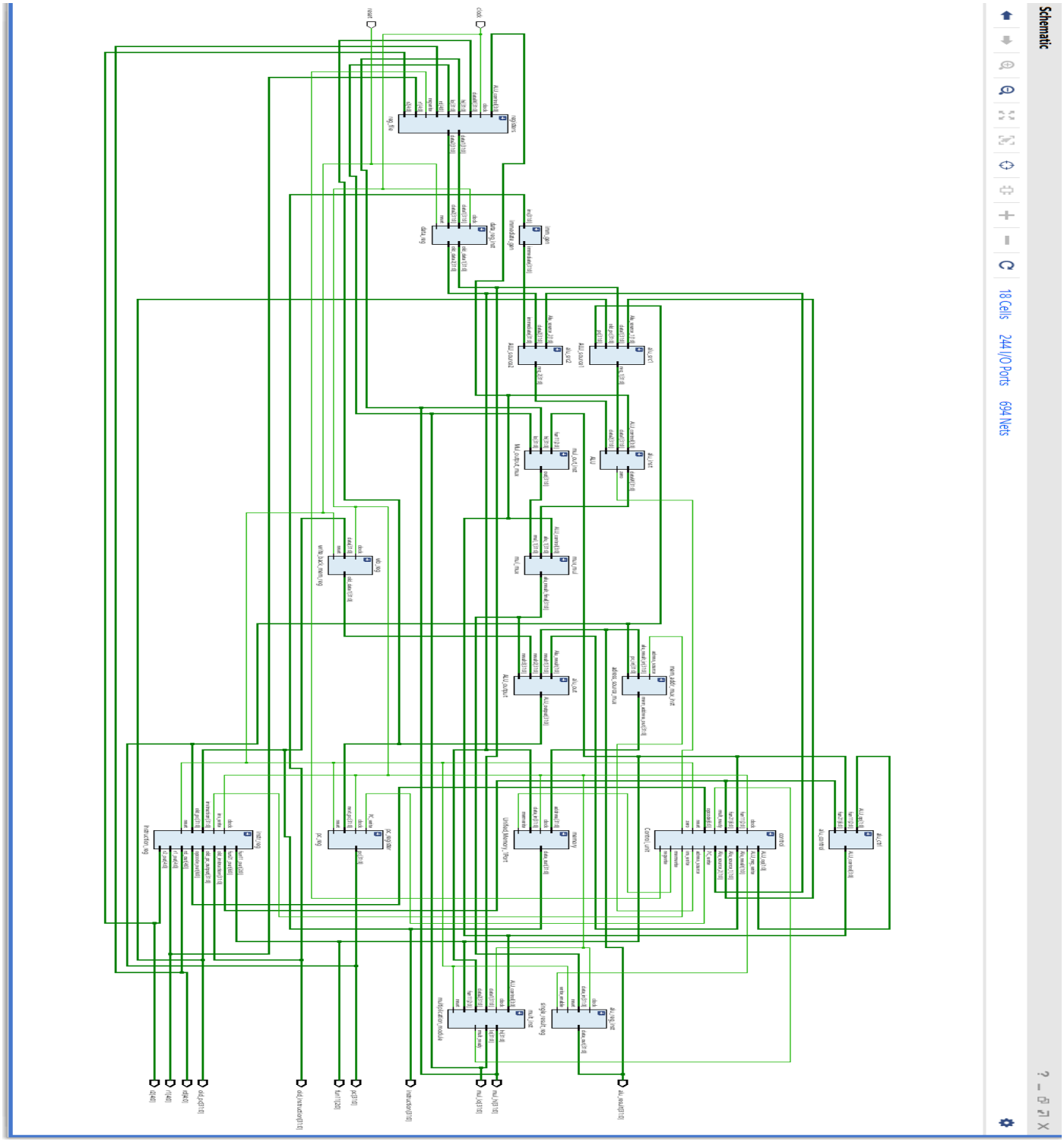
mulh x6, x5, x4

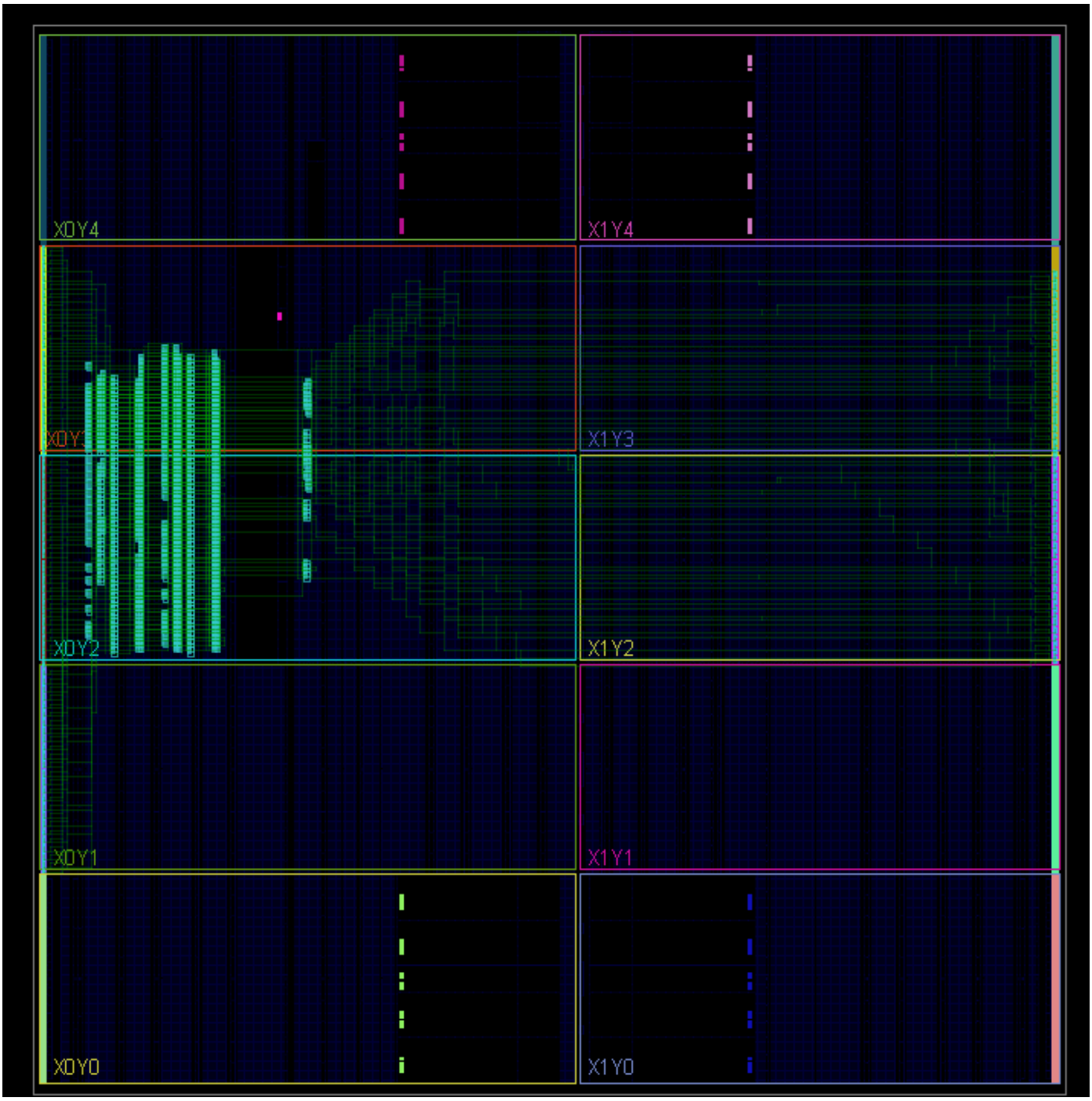
mulhsu x6, x5, x4

mulhu x6, x5, x4

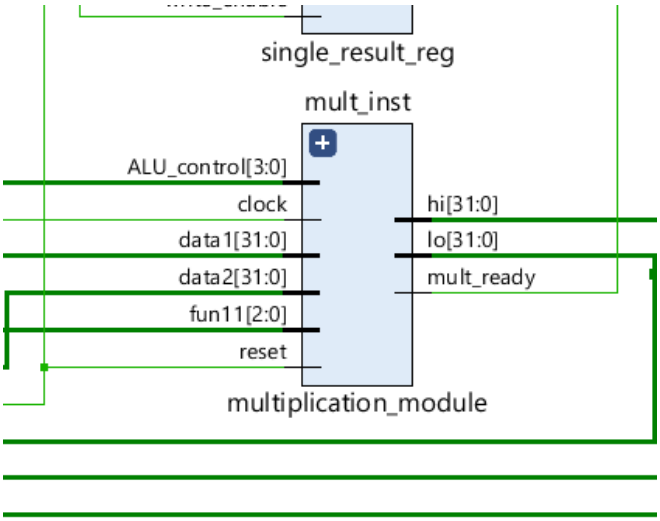
The waveform shows that it takes 32 cycles for multiplication and division, and the results are correct

The RTL Diagram

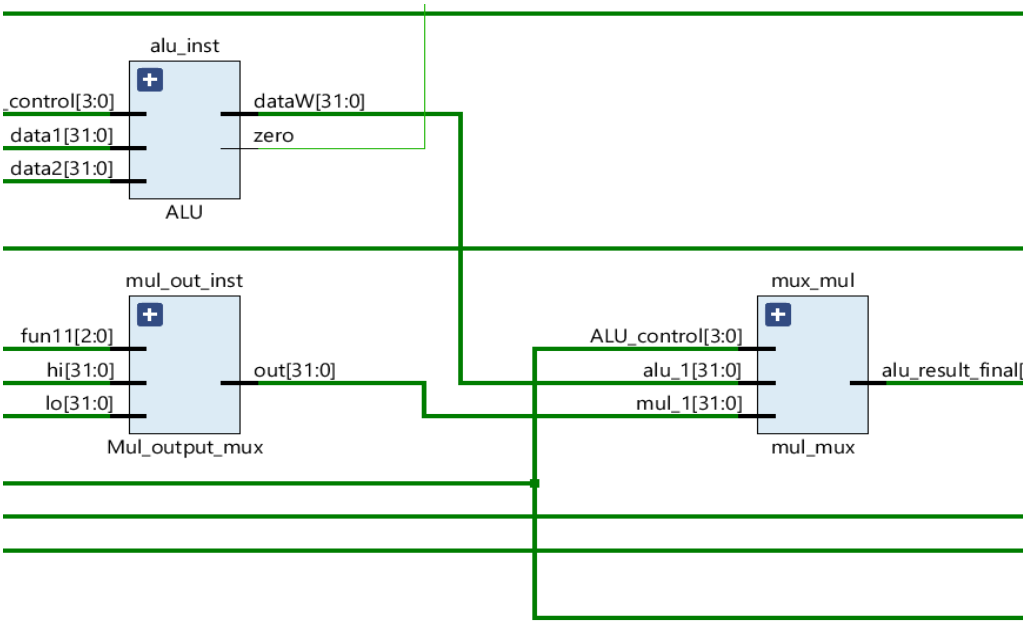



X0Y4
X1Y4
X0Y3
X1Y3
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0

The Multiplication module



This shows how the ALU and the multiplication results are connected and how the mux chooses between High and low, and then chooses the result from either the ALU or the multiplication module



Conclusion

- The project successfully fulfilled its primary objective of enhancing an integer-only RISC-V core with hardware-level multiplication and division capabilities. Key takeaways from the implementation include:
- Architectural Efficiency: The use of a multi-cycle architecture allowed for efficient resource sharing and higher clock frequencies by determining the clock period based on the slowest functional unit rather than the entire path.
- Effective Control Logic: The modification of the Main FSM to include the ExecM (S11) state effectively managed the high-latency operations of the co-processor, stalling execution until the mult_ready signal was received.
- Verified Performance: Simulation results confirmed that the multiplication_module correctly handles 32-bit operands, producing accurate 64-bit results across 32 clock cycles.
- Seamless Integration: Through the use of dedicated multiplexers (mul_mux and Mul_output_mux), the system maintains structural integrity, allowing the processor to switch dynamically between standard ALU results and high-speed arithmetic outputs.
- Overall, this hardware acceleration successfully mitigates the computational bottlenecks associated with software-defined arithmetic, providing a robust